Custom Deep Neural Network Architectures: From-Scratch Design, Implementation, and Evaluation

Across Classification, Regression, Forecasting, and Anomaly Detection

Bellatreche Mohamed Amine

April 28, 2026

1 Introduction

This report documents the design, implementation, and evaluation of three novel neural network architectures built entirely from scratch using primitive tensor operations. Rather than relying on standard library modules (`nn.Linear`, `nn.BatchNorm1d`), every layer is constructed directly with `nn.Parameter` and explicit matrix equations. The goal was to go beyond standard MLPs and ResNets and explore structurally different computation patterns for tabular data.

We evaluated these architectures across all 10 dashboard tasks spanning four categories:

- **Classification:** Heart disease, COVID-19, Weather (4-class)
- **Regression:** Temperature, Multi-output (Pressure + Humidity)
- **Forecasting:** Wind turbine power, Energy consumption
- **Anomaly detection:** Employee attrition, Heart disease, Wine type

All architectures were compared against the existing ML baselines (Stacking, XGBoost, Ridge, etc.) and our previous DNN results (sklearn MLP search, PyTorch standard, PyTorch advanced with ResidualMLP).

2 Primitive Layer Implementation

Before describing the architectures, we document the building blocks shared by all three. These are implemented in `pytorch_scratch_layers.py`:

- `MyLinear`: $\mathbf{y} = \mathbf{x}\mathbf{W}^\top + \mathbf{b}$ with explicit `nn.Parameter` for weight and bias. Kaiming He initialisation.
- `MyBatchNorm1d`: Explicit train/eval normalisation with learnable γ, β and running-mean/variance buffers updated with momentum 0.1.

No `nn.Linear` or `nn.BatchNorm1d` is used anywhere in our novel architectures. This satisfies the "from scratch" requirement while still leveraging PyTorch's autograd for backpropagation.

3 Novel Architecture Designs

3.1 Architecture 1: Feature Crossing Network (Gating)

This was our first novel architecture. Each block splits processing into two parallel paths:

1. **Main path:** $\mathbf{h} = \text{ReLU}(\text{BN}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1))$
2. **Gate path:** $\mathbf{g} = \sigma(\text{BN}(\mathbf{W}_2\mathbf{x} + \mathbf{b}_2))$
3. **Crossing:** $\mathbf{c} = \mathbf{h} \odot \mathbf{g}$ (element-wise multiplication)
4. **Projection:** $\mathbf{o} = \text{ReLU}(\text{BN}(\mathbf{W}_3\mathbf{c} + \mathbf{b}_3)) + \mathbf{x}$

The gating mechanism allows the network to selectively pass or suppress features based on learned patterns. The sigmoid gate produces values in $[0, 1]$ that act as soft feature selectors, giving the network the ability to model feature interactions through multiplicative composition rather than additive stacking.

What makes it different from MLP: Standard MLPs process inputs through a single linear→activation chain. Our Feature Crossing block uses two independent projections fused by multiplication, creating second-order feature interactions that a single linear layer cannot represent.

3.2 Architecture 2: Squeeze-and-Excite Tabular Network (Channel Attention)

Inspired by channel attention concepts but redesigned for tabular 1-D data:

1. **Main transform:** $\mathbf{h} = \text{ReLU}(\text{BN}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1))$
2. **Squeeze:** $\mathbf{s} = \text{ReLU}(\mathbf{W}_s\mathbf{h} + \mathbf{b}_s)$ where $\mathbf{W}_s \in R^{(d/r)\times d}$ compresses to a bottleneck
3. **Excite:** $\mathbf{e} = \sigma(\mathbf{W}_e\mathbf{s} + \mathbf{b}_e)$ expands back and produces per-feature importance weights
4. **Recalibrate:** $\mathbf{o} = \mathbf{h} \odot \mathbf{e} + \mathbf{x}$

The bottleneck reduction ratio $r = 4$ forces the network to learn a compact summary of which features matter, then broadcast that back to recalibrate the full representation. This is fundamentally different from standard attention (no Q/K/V matrices) and from Feature Crossing (the gate here depends on the transformed features, not the raw input).

What makes it different: The SE block learns *per-sample, per-feature importance weights* through a compress-expand bottleneck. An MLP treats all features uniformly; a ResNet adds skip connections but doesn't re-weight features. The SE block adaptively amplifies or suppresses each hidden feature based on what the network has learned about the current input.

3.3 Architecture 3: Multi-Scale Feature Pyramid Network

This architecture processes each input at three different resolutions simultaneously:

1. **Small branch:** $\mathbf{s} = \text{ReLU}(\text{BN}(\mathbf{W}_s\mathbf{x}))$ where $\mathbf{W}_s \in R^{(d/4)\times d}$
2. **Medium branch:** $\mathbf{m} = \text{ReLU}(\text{BN}(\mathbf{W}_m\mathbf{x}))$ where $\mathbf{W}_m \in R^{(d/2)\times d}$
3. **Large branch:** $\mathbf{l} = \text{ReLU}(\text{BN}(\mathbf{W}_l\mathbf{x}))$ where $\mathbf{W}_l \in R^{d\times d}$
4. **Concatenate:** $\mathbf{f} = [\mathbf{s}; \mathbf{m}; \mathbf{l}]$
5. **Merge:** $\mathbf{o} = \text{ReLU}(\text{BN}(\mathbf{W}_f\mathbf{f})) + \mathbf{x}$

Multi-scale processing is well-established in computer vision (Feature Pyramid Networks, U-Net) but rarely applied to tabular data. The intuition: the small branch captures coarse global patterns using fewer parameters, the medium branch handles mid-level interactions, and the large branch preserves fine-grained feature details. The concatenation and merge step lets the network learn the optimal combination of all three resolution levels.

What makes it different: No other architecture in our experiments processes features at multiple widths in parallel. MLP, ResNet, FeatureCrossing, and SE blocks all use a single hidden dimension. Multi-Scale gives the network access to representations at different granularities simultaneously.

4 Training Configuration

All architectures were trained with:

Parameter	Value
Optimiser	AdamW (decoupled weight decay)
Learning rate	0.01
Weight decay	10^{-4}
Batch size	64
Max epochs	300
Early stopping patience	20 epochs
Initialisation	Kaiming He (via MyLinear)
Loss (classification)	BCELoss / CrossEntropyLoss
Loss (regression)	MSELoss

For each architecture, we tested two configurations:

- **Wide-shallow:** hidden_dim=128, num_blocks=2
- **Narrow-deep:** hidden_dim=64, num_blocks=3

5 Results

Note: The results table below will be populated after running the full training script. Run:

```
.venv\Scripts\python.exe train_all_novel_architectures.py
```

Results are saved to `pytorch_results/novel_architectures_all_results.csv`.

5.1 Detailed Results by Task

Heart Classification

Architecture	Test Metric	Val Metric	Epochs	Time (s)
FeatureCross_64x3	0.9393	0.9580	23	9.1
SqueezeExcite_128x2	0.9393	0.9630	26	5.2
MultiScale_64x3	0.9391	0.9546	27	9.0
MultiScale_128x2	0.9354	0.9554	25	7.1
SqueezeExcite_64x3	0.9325	0.9567	23	4.5
FeatureCross_128x2	0.9317	0.9560	23	10.7
Previous best DNN	0.9517	—	—	—
ML baseline (Stacking)	0.9784	—	—	—

COVID Classification

Architecture	Test Metric	Val Metric	Epochs	Time (s)
FeatureCross_128x2	0.8747	0.8646	21	8.1
MultiScale_128x2	0.8721	0.8985	22	10.7
SqueezeExcite_128x2	0.8699	0.8822	22	5.8
SqueezeExcite_64x3	0.8682	0.8762	24	9.6
MultiScale_64x3	0.8654	0.8620	25	12.3
FeatureCross_64x3	0.8529	0.8604	22	9.3
Previous best DNN	0.9009	—	—	—
ML baseline (Stacking (LR))	0.8975	—	—	—

Temperature Regression

Architecture	Test Metric	Val Metric	Epochs	Time (s)
MultiScale_128x2	0.9996	0.9996	115	300.5
FeatureCross_64x3	0.9996	0.9996	58	165.2
FeatureCross_128x2	0.9996	0.9995	56	140.4
SqueezeExcite_64x3	0.9994	0.9994	71	128.5
MultiScale_64x3	0.9994	0.9993	78	297.0
SqueezeExcite_128x2	0.9991	0.9991	44	68.1
Previous best DNN	0.7520	—	—	—
ML baseline (Stacking)	0.7766	—	—	—

Multi-Output Regression

Architecture	Test Metric	Val Metric	Epochs	Time (s)
MultiScale_128x2	0.3686	0.3879	86	238.1
SqueezeExcite_64x3	0.3593	0.3773	65	113.4
SqueezeExcite_128x2	0.3589	0.3787	85	144.6
FeatureCross_128x2	0.3568	0.3757	92	255.7
MultiScale_64x3	0.3543	0.3710	50	164.1
FeatureCross_64x3	0.3262	0.3566	84	251.8
Previous best DNN	0.4665	—	—	—
ML baseline (XGBoost)	0.9282	—	—	—

Weather Classification

Architecture	Test Metric	Val Metric	Epochs	Time (s)
FeatureCross_64x3	0.7667	0.5214	94	333.9
SqueezeExcite_128x2	0.7595	0.4869	61	122.5
MultiScale_128x2	0.7578	0.4919	72	258.3
FeatureCross_128x2	0.7550	0.4948	34	105.5
MultiScale_64x3	0.7545	0.4887	47	205.8
SqueezeExcite_64x3	0.7544	0.4791	53	121.2
Previous best DNN	0.6407	—	—	—
ML baseline (Random Forest)	0.8493	—	—	—

Wind Forecasting

Architecture	Test Metric	Val Metric	Epochs	Time (s)
MultiScale_128x2	0.9593	0.9703	83	371.8
FeatureCross_64x3	0.9585	0.9712	76	336.1
FeatureCross_128x2	0.9583	0.9713	72	273.7
SqueezeExcite_128x2	0.9581	0.9700	71	176.6
MultiScale_64x3	0.9577	0.9702	89	484.6
SqueezeExcite_64x3	0.9573	0.9698	55	157.2
Previous best DNN	0.9710	—	—	—
ML baseline (Ridge)	0.9714	—	—	—

Energy Forecasting

Architecture	Test Metric	Val Metric	Epochs	Time (s)
FeatureCross_128x2	0.9965	0.9967	87	414.4
FeatureCross_64x3	0.9961	0.9970	141	782.5
SqueezeExcite_64x3	0.9949	0.9952	67	238.3
MultiScale_128x2	0.9938	0.9941	77	430.7
SqueezeExcite_128x2	0.9932	0.9937	77	238.8
MultiScale_64x3	0.9930	0.9934	56	382.3
Previous best DNN	0.9977	—	—	—
ML baseline (Linear Reg.)	0.9985	—	—	—

Anomaly Employee

Architecture	Test Metric	Val Metric	Epochs	Time (s)
MultiScale_64x3	0.8379	0.8390	32	419.5
SqueezeExcite_64x3	0.8373	0.8392	34	235.5
FeatureCross_128x2	0.8359	0.8376	28	260.8
FeatureCross_64x3	0.8356	0.8391	29	312.6
MultiScale_128x2	0.8353	0.8382	29	316.9
SqueezeExcite_128x2	0.8332	0.8384	26	160.4
Previous best DNN	0.7423	—	—	—
ML baseline (DBSCAN)	0.4694	—	—	—

Anomaly Heart

Architecture	Test Metric	Val Metric	Epochs	Time (s)
SqueezeExcite_128x2	0.9462	0.9558	25	3.3
FeatureCross_128x2	0.9396	0.9531	38	7.1
MultiScale_128x2	0.9390	0.9497	24	5.3
FeatureCross_64x3	0.9349	0.9483	22	5.2
MultiScale_64x3	0.9301	0.9527	44	11.6
SqueezeExcite_64x3	0.9238	0.9631	28	4.1
Previous best DNN	0.7711	—	—	—
ML baseline (Elliptic Env.)	0.6640	—	—	—

Anomaly Wine

Architecture	Test Metric	Val Metric	Epochs	Time (s)
FeatureCross_128x2	1.0000	0.9953	53	53.1
MultiScale_64x3	0.9999	0.9959	20	29.0
SqueezeExcite_64x3	0.9998	0.9958	42	31.6
MultiScale_128x2	0.9995	0.9956	24	29.0
FeatureCross_64x3	0.9978	0.9941	20	24.1
SqueezeExcite_128x2	0.9973	0.9954	20	13.5
Previous best DNN	0.9937	—	—	—
ML baseline (Elliptic Env.)	0.9188	—	—	—

5.2 Grand Summary: Best Novel Architecture per Task

Task	Metric	Best Novel Arch.	Novel	Prev DNN	ML	Best?
Heart Classification	ROC-AUC	FeatureCross_64x3	0.9393	0.9517	0.9784	ML
COVID Classification	AUC-ROC	FeatureCross_128x2	0.8747	0.9009	0.8975	Prev DNN
Temperature Regression	R^2	MultiScale_128x2	0.9996	0.7520	0.7766	Novel
Multi-Output Regression	Avg R^2	MultiScale_128x2	0.3686	0.4665	0.9282	ML
Weather Classification	AUC	FeatureCross_64x3	0.7667	0.6407	0.8493	ML
Wind Forecasting	R^2	MultiScale_128x2	0.9593	0.9710	0.9714	ML
Energy Forecasting	R^2	FeatureCross_128x2	0.9965	0.9977	0.9985	ML
Anomaly Employee	F1	MultiScale_64x3	0.8379	0.7423	0.4694	Novel
Anomaly Heart	Bal. Acc.	SqueezeExcite_128x2	0.9462	0.7711	0.6640	Novel
Anomaly Wine	F1	FeatureCross_128x2	1.0000	0.9937	0.9188	Novel

6 Existing Results Summary (Pre-Novel Architectures)

For reference, here are all results from our previous experiments that the novel architectures are compared against.

6.1 sklearn MLP Search (15 configs per task)

Task	Metric	ML	DNN	Δ	Winner
Heart classification	ROC-AUC	0.9784	0.9416	−0.0368	ML
COVID classification	AUC-ROC	0.8975	0.9009	+0.0034	DNN
Temperature regression	R^2	0.7766	0.7439	−0.0327	ML
Multi-output regression	Avg R^2	0.9282	0.4665	−0.4617	ML
Weather classification	AUC	0.8493	0.6407	−0.2086	ML
Wind forecasting	R^2	0.9714	0.9706	−0.0008	ML
Energy forecasting	R^2	0.9985	0.9977	−0.0008	ML
Anomaly (Employee)	F1	0.4694	0.7423	+0.2729	DNN
Anomaly (Heart)	Bal. Acc.	0.6640	0.7711	+0.1071	DNN
Anomaly (Wine)	F1	0.9188	0.9937	+0.0749	DNN

6.2 PyTorch Standard + Advanced (Heart, Temperature, Wind)

Task	ML	sklearn DNN	PyTorch std.	PyTorch adv.	Gap to ML
Heart (AUC)	0.9784	0.9416	0.9485	**0.9517**	−0.0267
Temp (R^2)	0.7766	0.7439	0.7486	**0.7520**	−0.0246
Wind (R^2)	0.9714	0.9706	**0.9710**	n/a	−0.0004

6.3 Previous Scratch Layers (ScratchFlexMLP, ScratchResidualMLP)

Task	pt_adamw	adv_residual_wide	Scratch best	Overall best
Heart (AUC)	0.9485	**0.9517**	0.9420	0.9517
Temperature (R^2)	0.7486	**0.7520**	0.7347	0.7520
Wind (R^2)	0.9708	n/a	0.9710	**0.9710**

7 Architecture Comparison and Analysis

7.1 Structural Differences Between All Architectures Tested

Architecture	Type	Key Mechanism	From Scratch?
sklearn MLP	Standard MLP	Linear → activation stack	No (sklearn)
PyTorch FlexMLP	Standard MLP	Same, with PyTorch training	Partial (nn.Linear)
PyTorch ResidualMLP	ResNet-style	Skip connections + BN	Partial (nn.Linear)
ScratchFlexMLP	Standard MLP	Same as FlexMLP	Yes (MyLinear)
ScratchResidualMLP	ResNet-style	Same as ResidualMLP	Yes (MyLinear)
FeatureCrossing	**Novel: Gating**	**Dual-path + sigmoid gate + multiply**	**Yes (MyLinear)**
SqueezeExcite	**Novel: Attention**	**Bottleneck + per-feature weights**	**Yes (MyLinear)**
MultiScale	**Novel: Pyramid**	**3 parallel branches at diff. widths**	**Yes (MyLinear)**

The bottom three rows are our novel contributions. Each introduces a fundamentally different processing topology that does not exist in the standard MLP or ResNet families.

8 Conclusion

We designed and implemented three structurally novel neural network architectures from scratch, each introducing a different mechanism for processing tabular data:

1. **Feature Crossing** (gating): multiplicative feature interaction through dual-path processing
2. **Squeeze-and-Excite** (attention): per-feature importance recalibration through a bottleneck
3. **Multi-Scale Pyramid** (multi-resolution): parallel processing at different granularities

All three are built entirely from primitive layers (explicit parameter matrices and batch normalisation buffers), demonstrating both implementation depth and architectural originality. The full results across all 10 tasks, comparing against ML baselines, sklearn DNNs, and previous PyTorch experiments, are saved in `pytorch_results/novel_architectures_all_results.csv`.

Key files:

- Architecture code: `novel_architectures.py`
- Primitive layers: `pytorch_scratch_layers.py`
- Training script: `train_all_novel_architectures.py`
- Results: `pytorch_results/novel_architectures_all_results.csv`
- Summary: `pytorch_results/novel_architectures_summary.csv`